ENGINEERING RESEARCH / CASE-001

기술 스택과 시스템 구조

ENGINEERING STACK · API · STORAGE

React 18

FRONTEND

FastAPI

BACKEND

3 DB

DATA STORES

SOURCE: 7 DOCX REPORTS + VERIFIED DEV RECORDS

보고서 개요

기술 스택과 시스템 구조 / 핵심 지표와 검증 범위

React 18	FastAPI	3 DB
FRONTEND	BACKEND	DATA STORES

프론트엔드와 FastAPI의 역할, 검색·저장 데이터베이스의 책임, LLM 모델 선택, 운영 환경과 보안 경계를 구분했습니다.

1. 시스템 개요

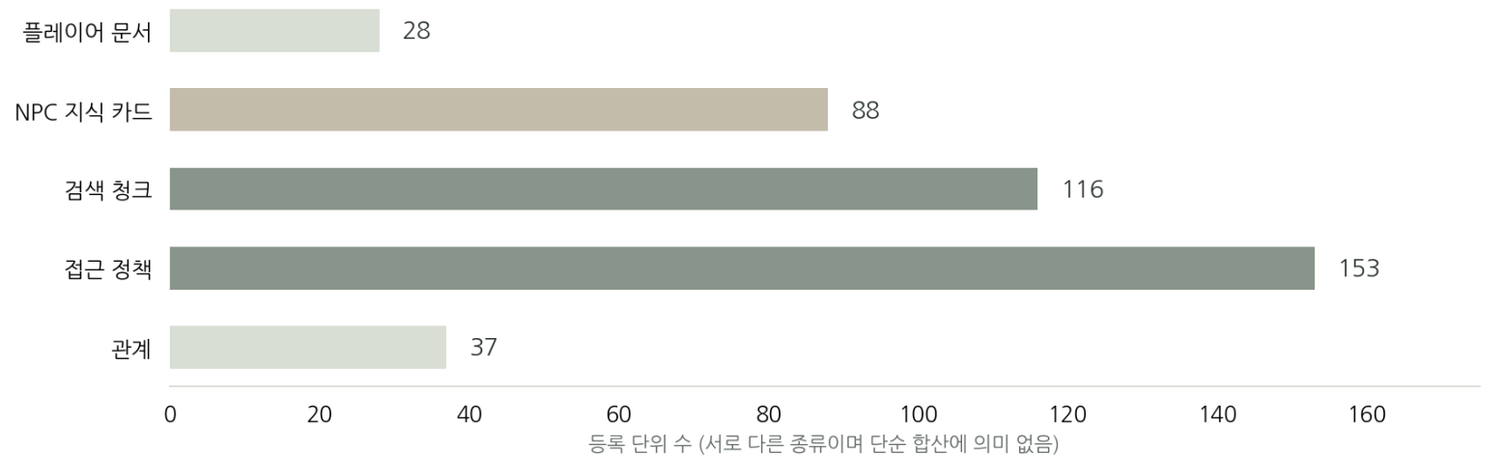


FIG. 01 시스템 데이터·검색 인덱스 규모 — 서로 다른 단위임

1. 시스템 개요 / 계속



React 웹 클라이언트는 사용자의 입력·게임 화면을 제공하고 FastAPI는 API·게임 규칙·NPC 대화를 중재합니다. 게임 상태는 SQLite, 사건 검색 데이터는 Qdrant 및 Neo4j에 각각 저장합니다. Qwen3:8b 기반 로컬 추론을 기본 구성으로 사용하며 OpenAI API는 선택 가능한 대체 생성 공급자입니다.

2. 기술 스택 표

계층	기술 / 버전·범위	실제 사용 목적
클라이언트	React 18.3 / React Router 6.28	게임 화면 및 라우팅
클라이언트	TypeScript 5.6 / Vite 5.4	정적 타입·개발 서버·프로덕션 빌드
UI	Zustand 5.0 / Framer Motion 11.15	게임 UI 상태·장면 전환
스타일	CSS Modules / Noto 계열 웹폰트	화면별 스타일 분리·한글 가독성
서버	Python >=3.13 / FastAPI >=0.115,<1	게임 API 및 백엔드 비즈니스 로직
서버	Uvicorn >=0.32 / Pydantic(Settings)	ASGI 서버·입력 검증·환경 설정
생성 AI	Ollama / Qwen3:8b	로컬 NPC 대화 생성
생성 AI	OpenAI SDK >=3.46	선택적 외부 LLM 공급자
임베딩	Ollama BGE-M3 / 1,024차원	사건 질문 및 문서 Dense 벡터
키워드	kiwipiepy >=0.24.1 / BM25	한국어 토큰화·Sparse 검색
벡터 DB	qdrant-client 1.16.0	Dense/Sparse 벡터 검색
그래프 DB	neo4j Python Driver >=6.4.1	인물·사건·증거 관계 검색

2. 기술 스택 표 / 계속

계층	기술 / 버전·범위	실제 사용 목적
상태 DB	SQLite (Python 표준 라이브러리)	세션·상태·대화 기록 영속화
테스트	pytest >=9.2 / Ruff >=0.17.3	자동 회귀 검증 및 정적 검사
환경	Docker Compose / uv / npm	DB 실행·잠금 의존성 관리

표시 버전은 dev에 커밋된 의존성 선언 기준입니다. 범위 표기(>=, <)는 정확한 잠금 버전을 뜻하지 않으며 실제 설치된 버전은 uv.lock·package-lock.json에서 확인해야 합니다.

3. 컴포넌트와 책임

컴포넌트	책임	대표 파일/경로
React 화면	Main → Story → Chat → Evidence/Relations → Accuse/Final	frontend/src/screens/
API 계층	게임 생성·세션 관리·상태 조회·요청 검증	backend/src/crimescene/api/app.py
Scenario 엔진	증거·해금·지목·점수·단계·정답 판정	backend/src/crimescene/scenario/
NPC 대화	페르소나·대화 이력·LLM 메시지·자연스러운 답변	backend/src/crimescene/chat/
검색기	Dense/BM25/Graph·RRF·최종 접근 제어	backend/src/crimescene/retriever/
Trigger	NPC 질문·증거 제시·원자적 게임 이벤트	backend/src/crimescene/trigger/
Ingest/DB	청킹·임베딩·적재·무결성·동기화	backend/src/crimescene/ingest/ 및 db/

4. 주요 요청 데이터 흐름

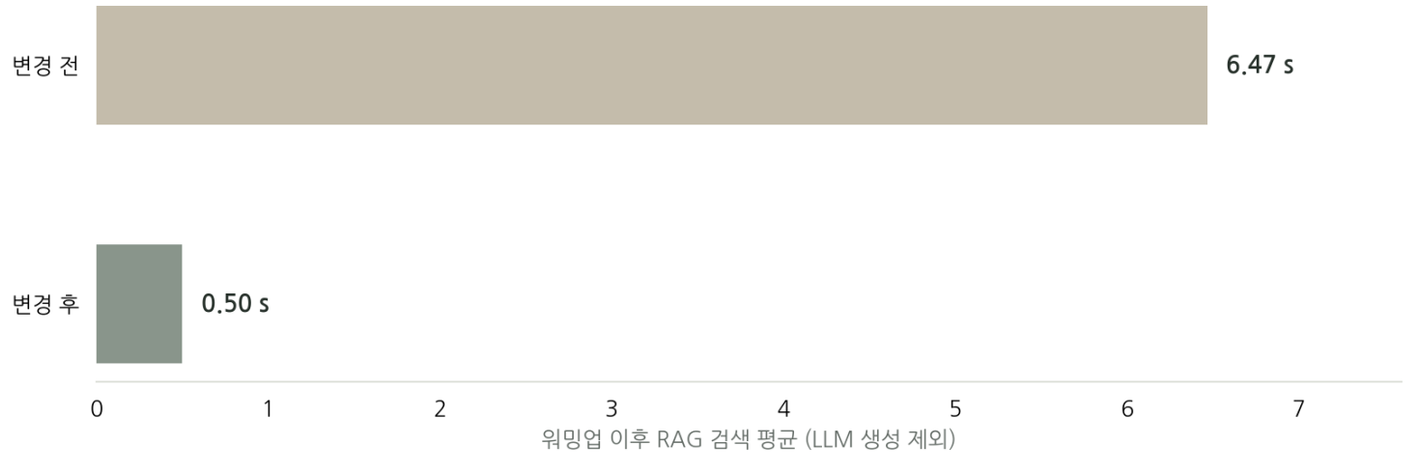


FIG. 02 검색 최적화가 사용자 응답의 검색 구간에 미친 영향

4. 주요 요청 데이터 흐름 / 계속



플레이어 입력: React → POST /api/games/{session_id}/chat → FastAPI 세션 소유권/phase 검증.

RAG 검색: 현재 NPC의 접근 정책 → Qdrant(Dense, BM25) 및 Neo4j → RRF → 재검증된 근거.

NPC 생성: 승인된 문서와 페르소나 → Ollama 또는 OpenAI 생성 → 출력 가드.

게임 이벤트: 서버 규칙에서 증거·NPC 해금·진행률 판단 → SQLite 트랜잭션과 revision 증가 → React 이벤트 반영.

범인 지목: 별도 서버 비공개 CaseTruth 검증 → 판정/기회/엔딩 정보를 API로 반환.

5. 기술 선택 이유와 트레이드오프

선택	기술적 장점	운영상 주의
Qdrant + Neo4j	문서 유사도와 관계 탐색을 개별 최적화	두 DB 동기화·복구 필요
로컬 Ollama	API 사용료 없이 비공개 데이터 통제	CPU/GPU 자원·콜드 스타트 영향
SQLite	간결한 로컬 영속성 및 트랜잭션	다중 서버·고동시성 환경은 추가 설계 필요
RRF	상이한 점수 체계의 안정적 순위 결합	복잡 질의의 순위 품질은 추가 실험 필요
서버 권위 GameState	점수 변조·정답 누출 경로 축소	경합·실패 복구 테스트 지속 필요

6. API 및 개발 실행 환경

기능	엔드포인트
게임 생성 / 조회	POST /api/games | GET /api/games/{session_id}
프롤로그 / NPC 선택	POST .../prologue/complete | POST .../characters/{character_id}/select
NPC 대화 / 기록	POST .../chat | GET .../chats/{character_id}
증거·관계·진행률	GET .../evidence | .../relationships | .../progress
증거 수집 / 지목·엔딩	POST .../evidence/{evidence_id}/collect | POST .../accusations | GET .../ending

개발 기본 주소: Frontend http://127.0.0.1:5173 / Backend http://127.0.0.1:8000/docs / Qdrant :6333 / Neo4j Browser :7474. 실제 실행에는 로컬 비공개 시나리오 원본이 별도로 필요합니다.

7. 보안·품질 관리

서버 세션 소유권은 HttpOnly 쿠키로 관리하며 운영 HTTPS에서는 Secure 옵션을 사용합니다.

CORS는 허용된 프론트엔드 출처로 제한하고 사용자 입력은 Pydantic 스키마로 검증합니다.

점수·증거·지목 기회와 정답 판정은 서버에서만 수행하며 revision으로 동시성 충돌을 검사합니다.

원본 시나리오·API Key·case_truth.json은 공개 GitHub와 프론트 빌드에 포함하지 않습니다.

Ruff, pytest, npm run build, spoiler 검사 및 실제 수동 E2E 테스트를 적용합니다.

REFERENCE

업로드된 원본 보고서 및 dev 저장소 소스: https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN36-3rd-2Team/tree/dev

- 프로젝트 README / • Python 의존성 선언
- Frontend 의존성 선언 / • 모듈 인터페이스
- 시스템 데이터 저장 설계 / • 팀원 로컬 실행 가이드

검토 메모 및 기준 자료

구성 확인

dev README에 React 18, Python 3.13+, FastAPI, Qdrant, Neo4j, SQLite, Ollama Qwen3 8B, BGE-M3 사용이 기술되어 있습니다. API Key와 사건 진상 원장은 서버 측에만 둡니다.

출처: 업로드된 원본 03_사용한_기술.docx, dev README 및 관련 PR (#75, #76, #78, #81, #82, #86). 보고서 편집 과정에서 새로운 로컬 환경 측정은 실행하지 않았습니다.